

Project 4: Shor Code for Quantum Error Correction

Prisha Maggu and Shiran Zhang

Princeton University

December 2025

ECE396/COS396: Introduction to Quantum Computing

This assignment was completed in accordance with University academic integrity policies.

/s/ Prisha Maggu and Shiran Zhang

1 Introduction and Background

The Shor code is one of the earliest and most representative in a series of coding schemes called **quantum error correcting codes**[1]. This series of coding schemes all have the final goal of correcting computation errors, often arising from coupling between the environment and the qubit. These interactions sometimes result in single qubit errors, such as random bit flips or phase changes, that alter the rest of the quantum algorithm.

These states are highly susceptible to noise and decoherence, leading to errors. Correcting these errors is non-trivial due to fundamental quantum mechanics principles: the no-cloning theorem and that measuring a qubit to check for an error destroys the quantum information it holds. Thus, any scheme that we use must take advantage of indirect measurements or entanglement. In short, the Shor code creates a large entangled 9-qubit state that, combined, contains the same logical information as before. By spreading the information across 9 qubits, this distribution immensely decreases the probability of error. Instead of measuring the logical qubit itself, the Shor Code for Quantum error correction uses auxiliary qubits and CNOT gates to measure the syndrome. A syndrome is defined as the pattern of measurement outcomes that indicates what error occurred and which qubit had that error, without revealing the logical quantum information being protected.

2 Task 1

2.1 Circuit Representation Sketch (Shor Encoding Circuit)

The Shor Code is a $[[9, 1, 3]]$ code, meaning it encodes one logical qubit into nine physical qubits and can correct up to one arbitrary single-qubit error. In particular, the Shor code $|\psi\rangle_L$ encodes one logical qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ into nine physical qubits.

The logical basis states for the Shor code are defined as:

$$|0\rangle_L = \frac{1}{\sqrt{8}}(|000 + 111\rangle) \otimes (|000 + 111\rangle) \otimes (|000 + 111\rangle)$$
$$|1\rangle_L = \frac{1}{\sqrt{8}}(|000 - 111\rangle) \otimes (|000 - 111\rangle) \otimes (|000 - 111\rangle)$$

By linearity, any arbitrary qubit state $|\psi\rangle$ is encoded as $|\psi\rangle_L = \alpha|0\rangle_L + \beta|1\rangle_L$.

The Shor Code can be conceptually broken into two layers of error protection or two main parts: Bit-flip protection and Phase-flip protection.

Phase encoding (the outer code) is performed via the following: the first qubit is phase-encoded into a superposition using Hadamard (H) and CNOT gates (as in the 3-qubit phase flip code).

Bit flip (the inner code) encoding is performed via the following: the resulting three entangled blocks are further bit-encoded (as in the 3-qubit bit flip code) using CNOT gates, creating three copies of the phase-encoded state.

Below are two equivalent representations of this described Shor encoding circuit.

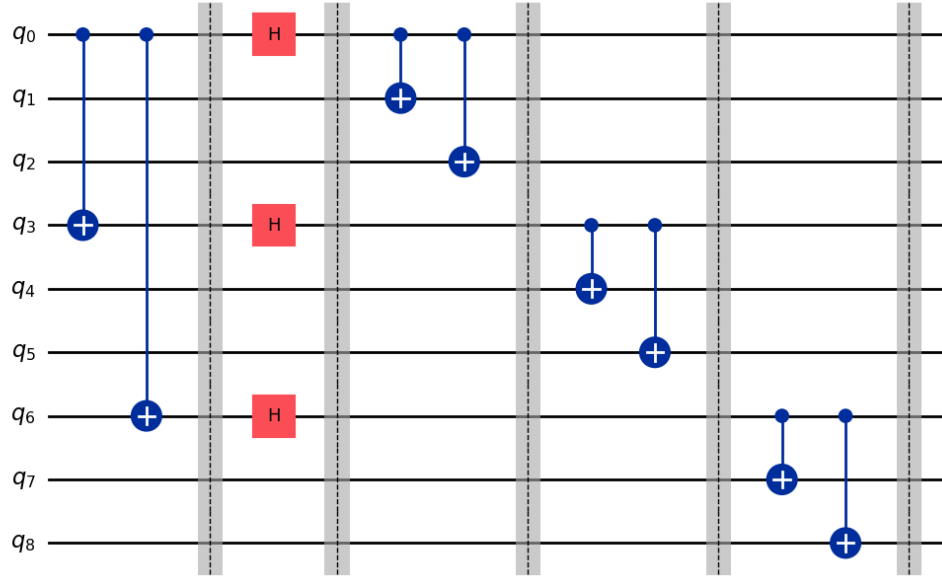


Figure 1: Circuit diagram representation of this encoding (in Colab)

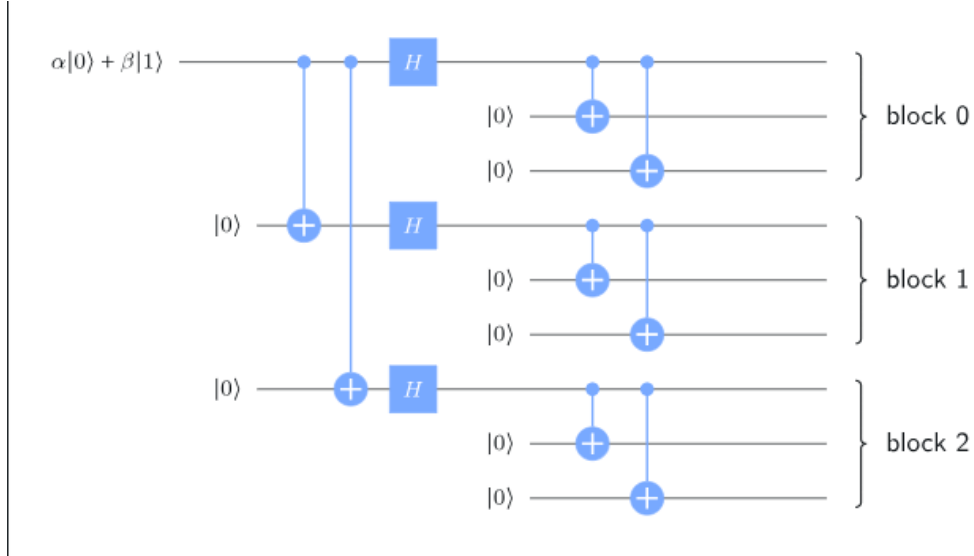


Figure 2: IBM Circuit diagram representation of this encoding

2.2 Correctness Proof of the Shor Code for Single-Qubit Error Correction

We will now prove that the Shor Code detects and corrects three single-qubit Pauli errors: Bit Flip (X), Phase Flip (Z), and a combination of both (Y). That is, the Shor code can detect and correct the errors $U = X$, $U = Z$, and $U = Y$ on any single qubit j . While we have explicitly shown the case for qubit 1, this proof is sufficient, as the stabilizer measurement and recovery procedure generalize to any qubit within the block, ensuring reliable detection and correction of single-qubit bit-flip or phase-flip errors.

Protection Against Bit-Flip Error ($U = X$)

Error: To detect the location of the bit-flip error, we measure the stabilizers S_{12} and S_{23} . The syndrome is determined by whether the error operator X_1 commutes or anti-commutes with each stabilizer.

Syndrome Measurement:

Action on $S_{12} = Z_1 Z_2$: Since X_1 anti-commutes with Z_1 ($X_1 Z_1 = -Z_1 X_1$) and commutes with $X_1 Z_2 = Z_2 X_1$, we examine the product $X_1 S_{12}$:

$$\begin{aligned}
X_1 S_{12} &= X_1 Z_1 Z_2 \\
&= (X_1 Z_1) Z_2 \\
&= (-Z_1 X_1) Z_2 \\
&= -Z_1 (X_1 Z_2) \\
&= -Z_1 (Z_2 X_1) \\
&= -(Z_1 Z_2) X_1 \\
&= -S_{12} X_1.
\end{aligned}$$

Since $X_1 S_{12} = -S_{12} X_1$, they anti-commute. Hence, the measurement of S_{12} yields the eigenvalue -1 .

Action on $S_{23} = Z_2 Z_3$: The error operator X_1 acts on qubit q_1 , while $Z_2 Z_3$ acts on qubits q_2 and q_3 . Since they act on disjoint sets of qubits, they commute. Thus, the measurement of S_{23} yields the eigenvalue $+1$.

The syndrome obtained from the stabilizer measurements is

$$(S_{12}, S_{23}) = (-1, +1).$$

This unique syndrome identifies a bit-flip error on qubit q_1 in the first block.

Correction: Once the location of the error is determined by the syndrome, a recovery operation R is applied. Since the error was $E = X_1$, the recovery operator is $R = X_1$. Applying the recovery operation restores the original logical state:

$$R(E|\psi\rangle_L) = X_1(X_1|\psi\rangle_L) = X_1^2|\psi\rangle_L = I|\psi\rangle_L = |\psi\rangle_L,$$

since $X_1^2 = I$. (Note: complete set of syndromes are detailed in 4.3)

Protection Against Phase Flip Error (U = Z)

Error: Suppose a phase-flip error Z acts on qubit q_1 of the encoded state, so that the corrupted logical state is

$$Z_1 |\psi\rangle_L.$$

Syndrome Measurement: To detect phase-flip errors, we measure the X -type stabilizers that compare the relative phase between blocks. The relevant stabilizers are

$$S_1 = X_1 X_2 X_3 X_4 X_5 X_6, \quad S_2 = X_4 X_5 X_6 X_7 X_8 X_9.$$

The error operator Z_1 anti-commutes with X_1 but commutes with all other X_j operators for $j \neq 1$.

Action on S_1 : Since Z_1 anti-commutes with X_1 , we have

$$\begin{aligned}
Z_1 S_1 &= Z_1 (X_1 X_2 X_3 X_4 X_5 X_6) \\
&= (Z_1 X_1) X_2 X_3 X_4 X_5 X_6 \\
&= (-X_1 Z_1) X_2 X_3 X_4 X_5 X_6 \\
&= -S_1 Z_1.
\end{aligned}$$

Thus, the measurement of S_1 yields the eigenvalue -1 .

Action on S_2 : The stabilizer S_2 acts on qubits 4 through 9, which are different from qubit 1. Therefore, Z_1 commutes with S_2 . Hence, the measurement of S_2 yields the eigenvalue $+1$. Thus, the measured syndrome is

$$(S_1, S_2) = (-1, +1).$$

This syndrome uniquely identifies a phase-flip error in Block 1. Although the stabilizers identify the affected block rather than the exact qubit, correcting any one qubit in the block restores the logical state.

Correction A suitable recovery operation is a phase flip applied to one qubit in the affected block. In this case, we apply

$$R = Z_1.$$

The recovery restores the logical state:

$$RZ_1 |\psi\rangle_L = Z_1 Z_1 |\psi\rangle_L = |\psi\rangle_L,$$

since $Z_1^2 = I$. (Note: complete set of syndromes are detailed in 4.3)

Protection Against Combined Errors ($U = XZ = iY$): The Pauli Y operator corresponds to a simultaneous bit-flip and phase-flip error, up to a global phase:

$$Y = iXZ.$$

Since a global phase has no physical significance, a Y error can be corrected by independently correcting its X and Z components. Suppose that a Y error acts on qubit q_1 , resulting in the corrupted logical state

$$Y_1 |\psi\rangle_L.$$

First, the bit-flip syndrome measurement is performed using the Z -type stabilizers within the block. The resulting syndrome uniquely identifies that a bit-flip error occurred on qubit q_1 . Next, the phase-flip syndrome measurement is performed using the $X^{\otimes 6}$ stabilizer checks. These measurements reveal that a phase-flip error occurred in Block 1. Since both the bit-flip and phase-flip components of the error have been identified, a recovery operation is applied. The appropriate recovery operator is

$$R = Y_1,$$

or equivalently,

$$R = X_1 Z_1.$$

Applying this recovery operation restores the original logical state.

Now, using our knowledge of the parameterization of 2×2 unitary matrices, we will argue that the Shor code can detect and protect against arbitrary single-qubit errors.

Using the parameterization of 2×2 unitary matrices, we argue that the Shor code can detect and correct an arbitrary single-qubit error. Any unitary error acting on a single qubit can be written as a linear combination of Pauli operators:

$$U_{\text{error}} = c_I I + c_X X + c_Y Y + c_Z Z,$$

where $c_I, c_X, c_Y, c_Z \in \mathbb{C}$. If this error acts on qubit q_1 of the encoded state, the resulting corrupted state is

$$|\psi_{\text{error}}\rangle = U_{\text{error}} \otimes I^{\otimes 8} |\psi\rangle_L = (c_I I + c_X X + c_Y Y + c_Z Z)_1 |\psi\rangle_L.$$

Syndrome measurements are projective measurements that are designed to commute with the logical operators but to either commute or anti-commute with the Pauli error operators X , Y , and Z . As a result, measuring the stabilizers projects the corrupted state onto one of the Pauli error subspaces corresponding to I , X , Y , or Z acting on a specific qubit (corresponding to a single, localized Pauli error P_j). The state gets projected onto an eigen-subspace of the stabilizers. The syndrome reveals the type of error and localizes it to the appropriate qubit or block, which is sufficient for recovery. Once this information is obtained, an appropriate recovery operation R is applied, restoring the original logical state. In particular, an appropriate deterministic recovery operation $R = P_j$ is then applied, which successfully restores the original logical state:

$$R|\psi_{\text{projected}}\rangle = P_j(P_j|\psi\rangle_L) = I|\psi\rangle_L = |\psi\rangle_L$$

Thus, the Shor code protects against arbitrary single-qubit errors.

3 Task 2

Implementation Overview of Shor's Code Using Qiskit:

3.1 Initialization.

In Qiskit, we initialize a nine-qubit register to the state

$$|\psi\rangle |00000000\rangle,$$

where the first qubit contains the state $|\psi\rangle$ to be protected and the remaining eight qubits are initialized to $|0\rangle$.

3.2 Encoding Unitary.

An encoding unitary U_{Shor} is implemented such that

$$U_{\text{Shor}}(|\psi\rangle |00000000\rangle) = |\psi\rangle_L.$$

This unitary is constructed in two stages: the phase-flip encoding (the outer encoding) and the bit flip encoding (the inner encoding). Following our encoding circuit diagram representation from **Task 1**, this produces the nine-qubit Shor code state.

Listing 1: Shor Code Encoding Unitary

```
1 def create_shor_encoder():
2     qc = QuantumCircuit(9)
3
4     # Step 1: Create 3-qubit phase flip code
5     # "Outer Code"
6     qc.cx(0, 3)
7     qc.cx(0, 6)
8     qc.barrier()
9
10    qc.h(0)
11    qc.h(3)
12    qc.h(6)
13    qc.barrier()
14
15    # Step 2: Create bit flip code for each block of 3 qubits
16    # "Inner Code"
17    # Block 1 (qubits 0, 1, 2)
18    qc.cx(0, 1)
19    qc.cx(0, 2)
20    qc.barrier()
21
22    # Block 2 (qubits 3, 4, 5)
23    qc.cx(3, 4)
24    qc.cx(3, 5)
25    qc.barrier()
26
27    # Block 3 (qubits 6, 7, 8)
28    qc.cx(6, 7)
29    qc.cx(6, 8)
30    qc.barrier()
31
32    return qc
33
34 shor_circuit = create_shor_encoder()
35 shor_circuit.draw(output='mpl')
```

3.3 Test States.

To ensure that our implementation is correct, the encoding is tested for the following input states:

$$|0\rangle, \quad |1\rangle, \quad |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}},$$

$$|+i\rangle = \frac{|0\rangle + i|1\rangle}{\sqrt{2}}, \quad |-i\rangle = \frac{|0\rangle - i|1\rangle}{\sqrt{2}}.$$

In our implementation, these test states are initialized by the following section of code:

Listing 2: Initializing Test States

```

1 def prepare_initial_state(state_name):
2
3     qc = QuantumCircuit(1)
4
5     if state_name == '0':
6         pass
7     elif state_name == '1':
8         qc.x(0)
9     elif state_name == '+':
10        qc.h(0)
11    elif state_name == '-':
12        qc.x(0)
13        qc.h(0)
14    elif state_name == '+i':
15        qc.h(0)
16        qc.s(0)
17    elif state_name == '-i':
18        qc.h(0)
19        qc.sdg(0)
20    else:
21        raise ValueError(f"Unknown state: {state_name}")
22
23    return qc

```

3.4 Logical Operators.

The logical Pauli operators act on the two-dimensional codespace spanned by $\{|0_L\rangle, |1_L\rangle\}$. In the code, the logical operators are defined abstractly by their action on the logical basis states:

$$X_L = |0_L\rangle\langle 1_L| + |1_L\rangle\langle 0_L|,$$

$$Y_L = -i|0_L\rangle\langle 1_L| + i|1_L\rangle\langle 0_L|,$$

$$Z_L = |0_L\rangle\langle 0_L| - |1_L\rangle\langle 1_L|.$$

Listing 3: Initializing Test States

```

1 def create_shor_codewords():
2     # Take the Kronecker product of three such states to form 9-qubit Shor
3     codewords.
4     def basis(sign=+1):
5         vec = np.zeros(8, dtype=complex)
6         vec[0] = 1.0
7         vec[7] = sign
8         return vec / np.sqrt(2)

```

```

8
9     cat_plus = basis(+1)
10    cat_minus = basis(-1)
11
12    # 9-qubit logical codewords as shown above
13    zeroL = np.kron(cat_plus, np.kron(cat_plus, cat_plus))
14    oneL = np.kron(cat_minus, np.kron(cat_minus, cat_minus))
15    return zeroL, oneL
16
17 def create_logical_operator(pauli_op):
18     # Pauli operators act on the 2D logical subspace
19     zeroL, oneL = create_shor_codewords()
20
21     if pauli_op == 'X':
22         return np.outer(zeroL, oneL.conj()) + np.outer(oneL, zeroL.conj())
23     elif pauli_op == 'Y':
24         return -1j * np.outer(zeroL, oneL.conj()) + 1j * np.outer(oneL, zeroL.conj())
25     elif pauli_op == 'Z':
26         return np.outer(zeroL, zeroL.conj()) - np.outer(oneL, oneL.conj())
27     else:
28         raise ValueError(f"Unknown operator:{pauli_op}")

```

3.5 Expectation Values.

For each encoded test state $|\psi\rangle_L$, we compute the expectation values of the logical Pauli operators:

$$\langle X_L \rangle, \quad \langle Y_L \rangle, \quad \langle Z_L \rangle.$$

These are then compared with the expectation values of the corresponding single-qubit Pauli operators on the original state $|\psi\rangle$ (from the reference table):

$$\langle \psi | X | \psi \rangle, \quad \langle \psi | Y | \psi \rangle, \quad \langle \psi | Z | \psi \rangle.$$

This comparison confirms that the logical operators act on the encoded state in the same way as the Pauli operators act on the original single-qubit state[2].

3.6 Results

Expectation Values of Logical Operators for Shor Code From Our Code Output

Initial State	$\langle X_L \rangle$	$\langle Y_L \rangle$	$\langle Z_L \rangle$
$ 0\rangle$	+0.000	+0.000	+1.000
$ 1\rangle$	+0.000	+0.000	-1.000
$ +\rangle$	+1.000	+0.000	+0.000
$ -\rangle$	-1.000	+0.000	+0.000
$ +i\rangle$	+0.000	+1.000	+0.000
$ -i\rangle$	+0.000	-1.000	+0.000

Expected Single-Qubit Values for Reference

State	$\langle X \rangle$	$\langle Y \rangle$	$\langle Z \rangle$
$ 0\rangle$	0	0	+1
$ 1\rangle$	0	0	-1
$ +\rangle$	+1	0	0
$ -\rangle$	-1	0	0
$ +i\rangle$	0	+1	0
$ -i\rangle$	0	-1	0

4 Task 3

We want to look at the performance of the nine-qubit Shor code in a Pauli noise model. The goal of this task is to build on the previous sections, and implement a random noise model that applies Pauli X , Y , and Z with probability $p/3$ to each qubit independently. then, we want to see how well the Shor code can decode that state.

4.1 Pauli Noise Model

After initializing the qubits in the same way as in previous tasks, we first begin with the Noise model. For each qubit, we draw a random number $r \in [0, 1]$, then compare this number with p to determine which, if any, of the Pauli functions are applied. This ensures that with probability p , some noise is applied, and with probability $1 - p$ nothing happens. We set p as continuous later in this task. Mathematically, it was implemented as:

$$\begin{cases} X_i & r < p/3 \\ Y_i & p/3 \leq r < 2p/3 \\ Z_i & 2p/3 \leq r < p \\ I & r \geq p \end{cases} \quad (1)$$

4.2 Stabilizers and Syndromes

As explained before, error detection in the Shor code depends on stabilizer operators for bit-flip and phase-flip errors. We find stabilizer outcomes by computing the expectation values of the Pauli operators. They consist of:

- **Bit-flip stabilizers** within each block:

$$Z_0Z_1, Z_1Z_2, \quad Z_3Z_4, Z_4Z_5, \quad Z_6Z_7, Z_7Z_8$$

- **Phase-flip stabilizers** across blocks:

$$X_0X_1X_2X_3X_4X_5, \quad X_3X_4X_5X_6X_7X_8$$

We then evaluate stabilizers by computing the expectation values $\langle S \rangle = \langle \psi | S | \psi \rangle$. Each outcome is mapped to a syndrome of $+1$ or -1 .

4.3 Decoding

After some derivation, we can summarize all bit flip syndromes into a decoding rule:

(s_1, s_2)	Where the error is	Correction
$(+1, +1)$	None	I
$(-1, +1)$	First qubit	X
$(-1, -1)$	Middle qubit	X
$(+1, -1)$	Last qubit	X

Similarly for phase flip, we can write the decoding rule:

(t_0, t_1)	Identified block	Correction applied
$(+1, +1)$	None	I
$(-1, +1)$	$(0, 1, 2)$	Z
$(-1, -1)$	$(3, 4, 5)$	Z
$(+1, -1)$	$(6, 7, 8)$	Z

Using these rules, we can easily create a correction circuit block-by-block and apply it to the noisy state from before. This creates a complete decoding.

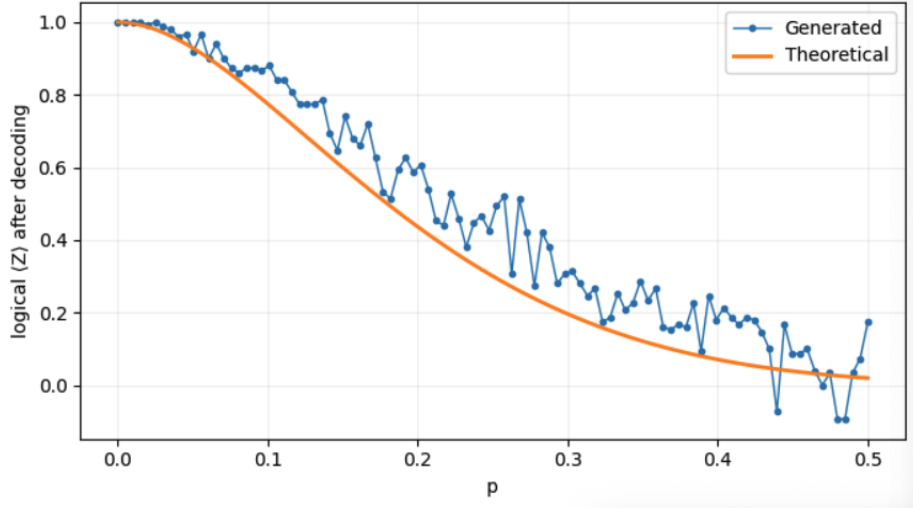


Figure 3: Final Logical Expectation Value vs p

4.4 Logical Z Expectation

At the end of this process we need to change the code back to a single qubit, and finally $\langle Z_L \rangle = \langle \psi_{\text{decoded}} | Z_0 | \psi_{\text{decoded}} \rangle$. If the value is close to +1, that means we were successful in the decode process. It is important to note the order of the qubits during this process, which took me a very long time to debug. In my code, I reordered the qubits from right to left to avoid crashes.

4.5 Monte Carlo Estimation and Theoretical Curve Confirmation

I used a Monte Carlo Estimation to create a plot as a function of p . We run a full-procedure of all the sections above N times. Arbitrarily, we have set $N = 300$. For p values, we run it from 0 to 0.5, with 100 samples to imitate a continuous curve.

As a consistency check, we compare the numerical results generated from the Monte Carlo data with an estimated theoretical curve. The actual curve is difficult to calculate within the scope of this project, so I made a simple approximation that the Shor code can perfectly correct any single qubit Pauli error, mathematically represented using the probability:

$$P_{\leq 1}(p) = (1 - p)^9 + 9p(1 - p)^8 \quad (2)$$

We note that this is not always true, and that the Shor code can correct some multi-qubit errors as well. This is why this theoretical curve serves as the **lower bound** only, and as a tool for us to analyze the major properties of the simulation.

4.6 Results

There are a few characteristics that we need to pay special attention to in our simulated results.

- At small p , we expect that the gradient should be almost flat. This is due to the Shor code's ability to correct the vast majority of single-qubit errors, which is the most common type in this range.
- At small p , the logical expectation is at +1, which shows successful decoding.
- Deviations at larger p comes from uncorrectable multi-qubit errors. The "lower bound" nature of the curve is reflected here.

Overall, this generated plot demonstrates the success of the Shor code in suppressing errors at low probabilities, and how it changes as the probability of error increases.

References

- [1] P. W. Shor, “Scheme for reducing decoherence in quantum computer memory,” *Phys. Rev. A* **52** (Oct, 1995) R2493–R2496. <https://link.aps.org/doi/10.1103/PhysRevA.52.R2493>.
- [2] IBM Quantum, “The 9-qubit shor code.” 2025. <https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/correcting-quantum-errors/shor-code>. Accessed: 2025-12-14.